



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS TRINDADE
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CURSO CIÊNCIAS DA COMPUTAÇÃO

Pedro Henrique Gimenez - 23102766
Rita Louro Barbosa – 22203157
Rafael Neves de Mello Oliveira - 17102816

INE5415 – Teoria da Computação

Trabalho 1



1. Máquinas de Turing Determinística de Fita Única.....	4
2. Máquinas de Turing Determinística Multifitas.....	5
3. Implemente Máquinas de Turing Multifitas.....	6



Abstract

Este documento tem como propósito a explicação das questões apresentadas no primeiro trabalho da disciplina de Teoria da Computação - INE5415. Cada questão será abordada com seu respectivo enunciado, resposta e uma explicação detalhada.



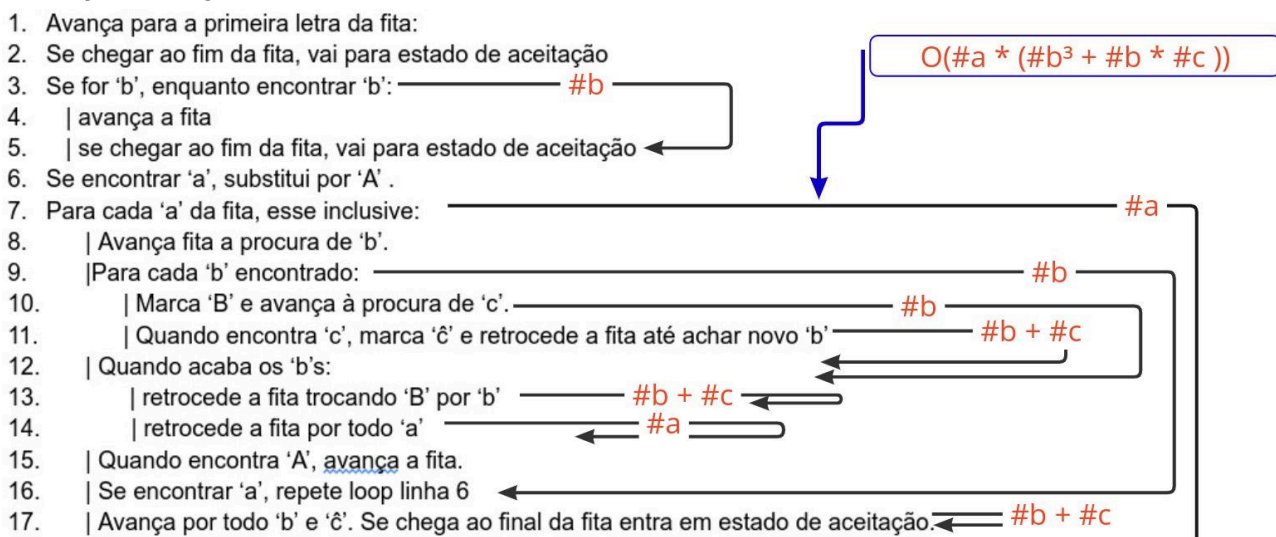
1. Máquinas de Turing Determinística de Fita Única

(a). Implemente Máquina de Turing Determinística com fita única para computar a seguinte linguagem:

$$L = \{a^i b^j c^k \mid i, j, k \in \mathbb{N} \text{ e } i \times j = k\}$$

Resposta:

Descrição do algoritmo:



Análise de complexidade:

Linha 1: 1 passo de computação

Linha 2 a 5: No máximo $\#b$

Linha 6: 1 passo de computação

Linha 7 a 17: No máximo $\#a$ passos

Linha 9 a 15: No máximo $\#b$ passos

Linha 10 a 12: No máximo $\#b$ passos

Linha 11: No máximo $\#b + \#c$ passos

Linha 14: No máximo $\#a$ passos

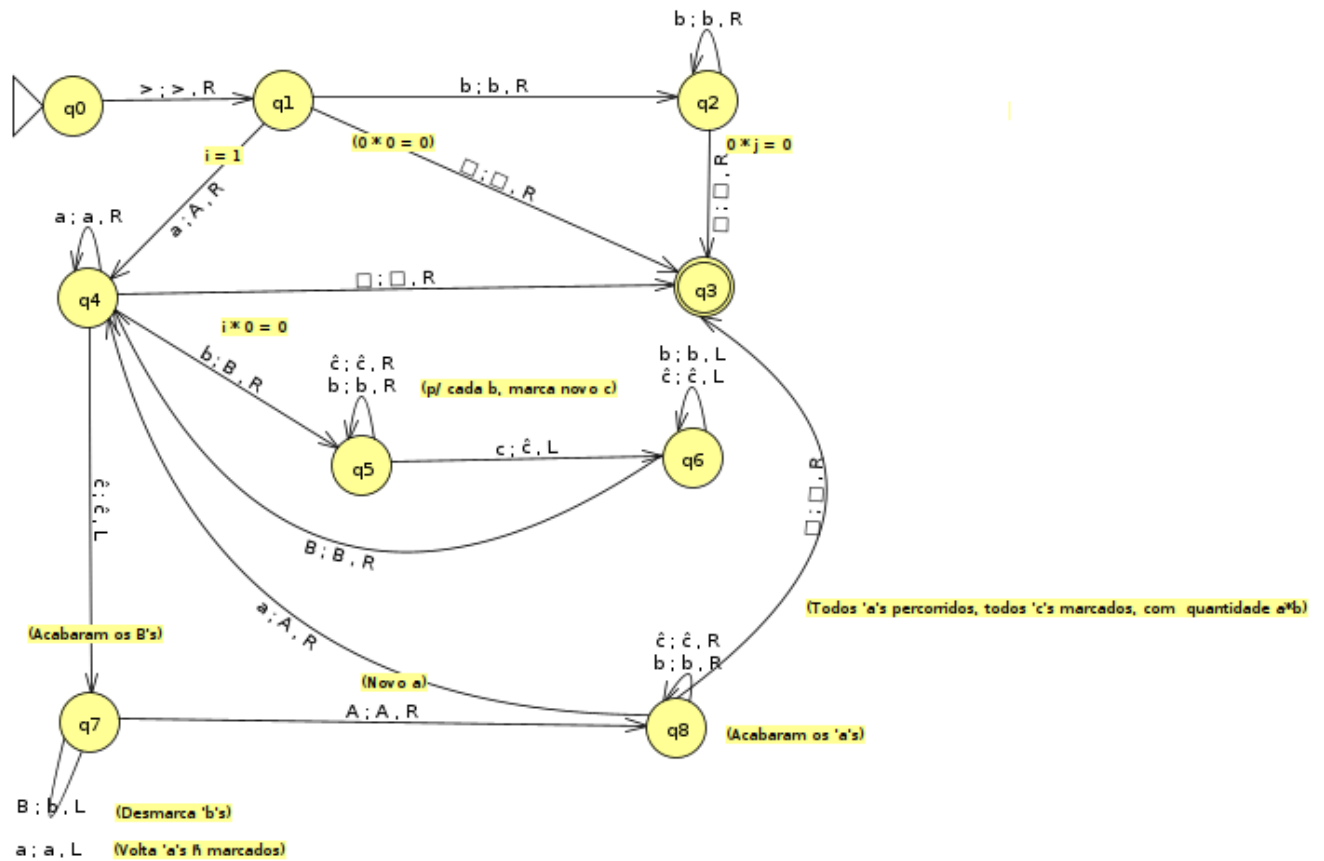
O maior loop(com mais passos internos) se dá na aplicação do procedimento de para cada 'a' na fita , procura cada b. para cada b, marca um novo 'c'. Isso requer um possível tempo exponencial de idas e vindas na fita a procura dos 'a's, 'b's, e 'c's.

Encadeando o maior loop, A função de complexidade total $T(n)$ é dada por:

$$\#a * \#b * (\#b + \#c) = O(\#a * (\#b^3 + \#b * \#c))$$



Lógica do autômato: Para cada "a", a MT marca 'b' * 'c's
 (Para cada 'a':
 Para cada 'b', marca B e marca um novo c.
 Quando acabam 'b's, desmarca todos os B's e marca o próximo "a", repetindo o processo.)





2. Máquinas de Turing Determinística Multifitas

Implemente Máquinas de Turing Multifitas para computar a seguinte linguagem:

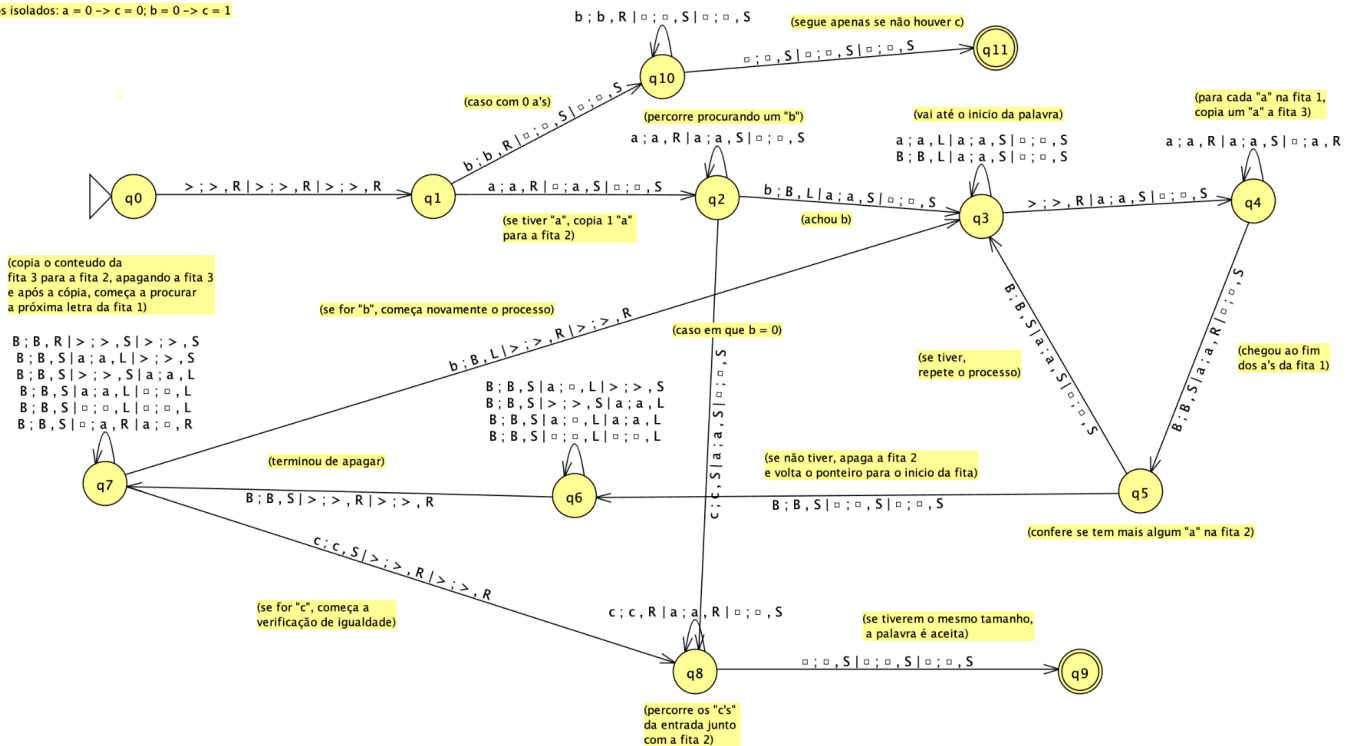
$$(a) L = \{a^i b^j c^k \mid i, j, k \in \mathbb{N} \text{ e } i^j = k\}$$

A entrada deve ser fornecida integralmente na primeira fita.

Resposta

Descrição do algoritmo:

1. Avança todas as fitas para a primeira posição de cada.
2. Confere se a palavra da entrada começa com **a** ou **b**.
 - a. se **b**, entra no caso 0^j , em que o número de **c**'s é igual a 0. Para isso, a máquina percorre todos os **b**'s e se chegar ao fim da fita 1 de entrada, a palavra é aceita.
 - b. se **a**, copia esse **a** para a fita 2 inicia o algoritmo de cálculo de exponenciação (descrito a partir do item 3).
3. Percorre a fita 1 passando por todos os **a**'s e **B**'s marcados até encontrar um **b** não marcado.
 - a. Se encontrar um **c**, ao invés de **b**, entra no caso i^0 , em que o número de **c**'s é igual a 1. Para isso, ela percorre os **c**'s da fita 1 junto à fita 2 (que atualmente possui um único **a**); se as duas fitas atingirem o fim ao mesmo tempo, a palavra é aceita.
 - b. Se encontrar um **b**, ela marca este **b** como visitado (**B**) e volta o ponteiro da fita 1 até encontrar o primeiro **a** (e continua com o algoritmo descrito no item 4).
4. Em seguida, para cada **a** encontrado na fita 1, a máquina copia um **a** para a fita 3.
5. Quando encontrar um **B**, significa que acabaram os **a**'s e então avança o ponteiro da fita 2.
 - a. Se encontrar um outro **a** na fita 2, repete o algoritmo a partir do item 3b.
 - b. Se encontrar vazio na fita 2, volta o ponteiro da fita 2 e da fita 3 até o início. Enquanto volta, apaga o conteúdo da fita 2. Após atingir o início das fitas 2 e 3, continua no item 6.
6. Com os ponteiros no início e a fita 2 limpa, a máquina percorre a fita 3, apagando e copiando seu conteúdo para a fita 2. Após atingir o fim da fita 3, a máquina retorna para o início das fitas 2 e 3 e, em seguida, avança a fita 1 até encontrar o próximo **b** não visitado ou um **c**.
 - a. Se encontrou um **b**, a máquina avança as fitas 2 e 3 para o primeiro slot e repete o algoritmo a partir do item 3b.
 - b. Se encontrou um **c**, significa que acabou a exponenciação e já pode conferir se a quantidade de **c**'s condiz com o calculado (descrito no item 7).
7. A máquina percorre cada **c** da fita 1 junto com cada **a** da fita 2; se as duas fitas atingirem o fim ao mesmo tempo, a máquina aceita a palavra e encerra o algoritmo.





3. Implemente Máquinas de Turing Multifitas

(a) O algoritmo de Euclides para o Máximo Divisor Comum. A máquina recebe como entrada uma sequência de símbolos representando n e m em representação unária. Ao término, a fita deve conter o $MDC(n, m)$.

Resposta

O algoritmo de Euclides $MDC(A, B)$: Se $B == 0$, A Se não, $MDC(B, A \text{ resto } B)$

- 1. Se ler 0 aceita
- 2. Avançar na fita por todo 1 até encontrar um 0
- 3. Para todo 1 lido depois do zero copie-o para a fita auxiliar.
- 4. Verificar qual fita ficou com o menor número menor numero(fita principal), maior(fita secundária)
- 5. Retroceder nas duas fitas, para cada 1 lido na fita fita principal, trocar por X e apague 1 da fita secundária.
- 6. Quando encontrar um vazio na fita principal, se a fita secundária estiver vazia, vá para o passo 7. Caso contrário, avançar nas duas fitas trocando todo X da fita principal por 1.
- 7. Quando encontrar um vazio, repetir do passo 3.
- 8. Retroceder na fita principal trocando todo X por 1 e aceitar.

Análise de complexidade

Considere a seguinte análise para o algoritmo com uma entrada unária A e B :

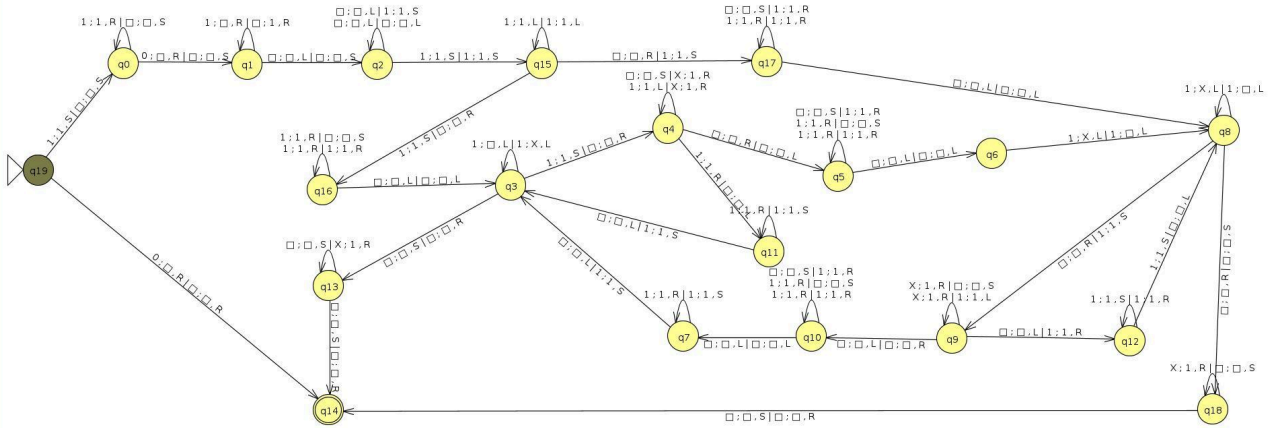
- 1. 1 passo de computação
- 2. n passos de computação
- 3. n passos de computação
- 4. $\min(A, B) \Rightarrow$ no máximo n passos de computação
- 5. $\min(A, B) \Rightarrow$ no máximo n passos de computação
- 6. $\min(A, B) \Rightarrow$ no máximo n passos de computação
- 7. $\log(n)$
- 8. $\min(A, B) \Rightarrow$ no máximo n passos de computação

Os passos 1 e 2 dependem da quantidade de símbolos representando cada número, sendo que pode levar até n passos de computação. Para verificar qual fita tem o menor número nos passos 3 e 4, é necessário comparar as duas sequências de "1" nas fitas principais e auxiliar. A comparação envolve a leitura sequencial de ambas as fitas, logo levará no máximo o tamanho da menor palavra ou seja complexidade linear de n passos. Cada iteração realiza uma operação similar à subtração sucessiva, o que é equivalente ao algoritmo de Euclides. Portanto, o número de iterações é limitado pelo número de vezes que é necessário subtrair B de A , logo a complexidade está em $\log(n)$.

A função de complexidade total $T(n)$ é dada por:



$$T(n) = n + n + n * \log(n) = O(n * \log(n))$$





(b) A série de Fibonacci. A máquina recebe como entrada uma sequência de símbolos que representa n (representação unária). Ao término, deve constar na fita uma sequência de símbolos que indica o valor do n -ésimo termo, ou seja $Fib(n)$. Exemplo: A máquina recebe como entrada a sequência "aaaaa"(5) e deve retornar "cccccccc"(8).

Resposta

- 1. Se está vazio, não faz nada e aceita.
- 2. Consome primeiro A da entrada e avança, se a entrada estiver vazia, aceita.
- 3. Consome segundo A da entrada e avança, escreve A na segunda fita, se a entrada estiver vazia, aceita.
- 4. Consome terceiro A da entrada e avança, escreve A na terceira fita, se a entrada estiver vazia, aceita.
- 5. Ler próximo A da entrada e somar a segunda fita com terceira fita.
- 6. Escrever o número anterior na terceira fita.
- 7. Consumir A da entrada e avançar, se vazio, aceita, se não, vá para o passo 5.

Análise de complexidade

Considere a seguinte análise para o algoritmo com uma entrada de tamanho n temos:

- 1. 1 passo de computação
- 2. 1 passo de computação
- 3. 1 passo de computação
- 4. 1 passo de computação
- 5. 1 passo de computação
- 6. 1 passo de computação
- 7. $n-3$ passos de computação

Os passos 1 até o passo 6 levam 1 passo de computação, portando complexidade constante. Já no passo 7 temos um laço que irá percorrer $n-3$ símbolos da palavra de entrada, logo complexidade n .

A função de complexidade total $T(n)$ é dada por:

$$T(n) = n + 3 = O(n)$$

